

CSCI 3271 — Operating Systems
Programming Project 1
Due: Friday September 4

Instructor: Edward Tremel
School of Computer and Cyber Sciences
Augusta University

Common Instructions: For this assignment you will need to write a program to solve a problem, based on handout code provided on D2L. You should upload your solution to D2L in individual plaintext files with the correct extension for source code (i.e. `.py` for Python, `.c` or `.h` for C). In general, you do not need to submit handout code files that are unchanged; you only need to upload code files that you modified in some way.

Project Description: Many of the OS algorithms we will study in this class rely on data structures such as linked lists, queues, and hashtables. However, the C programming language (in which all operating systems are written) does not include any data structures in its standard library; operating system writers must create them from scratch before they can use them. To give you some experience with this process, in this problem you will implement a basic vector in C. A vector is an array-like data structure with a flexible, rather than fixed, size; this data structure is called an `ArrayList` in Java and a `List` in C#, but in C++ it is called a vector, so we will use that name for it when implementing it in C.

The compressed folder `vector-cmake-project.zip`, available on D2L, contains 3 code files: `vector.h` provides the interface for the vector, `vector.c` provides the implementation of the vector functions, and `test_vector.c` shows how the vector might be used in a program. It also contains a file named `README.md` that describes how to compile and run the project, either with or without CMake.

You must write all of the (initially-empty) functions in `vector.c`, and provide a definition of the `struct vector` structure; you may also define any other structures or functions you want to use to support your implementation inside `vector.c`. Make sure your functions adhere to the interface described in the documentation comments in `vector.h`. Do not modify `vector.h`; you should submit only `vector.c`. To make sure your vector implementation works, you will probably want to add more code to `test_vector.c`, but you will not be graded on it.